

Object-Oriented Programming (OOP) dalam Python

Object-Oriented Programming atau OOP adalah paradigma pemrograman yang mengorganisasi kode ke dalam objek-objek. Setiap objek adalah instance dari sebuah class. Class adalah cetakan atau blueprint yang mendefinisikan attribute (data) dan method (fungsi) yang dimiliki objek.

Konsep Dasar OOP:

1. Class: cetakan untuk membuat objek. Ditulis dengan keyword 'class'.
2. Object: instance nyata dari sebuah class. Dibuat dengan memanggil nama class.
3. Attribute: variabel yang menyimpan data di dalam objek.
4. Method: fungsi yang didefinisikan di dalam class.

Contoh sederhana: class Mahasiswa memiliki attribute nama dan nim, serta method perkenalan().

Ketika kita membuat `mhs = Mahasiswa('Budi', '12345')`, mhs adalah object dari class Mahasiswa.

Kita bisa memanggil `mhs.perkenalan()` untuk menjalankan method-nya.

Empat Pilar OOP:

- Encapsulation: menyembunyikan detail internal, hanya ekspos interface yang diperlukan.
- Inheritance: class anak mewarisi attribute dan method dari class induk.
- Polymorphism: method yang sama bisa berperilaku berbeda di class yang berbeda.
- Abstraction: menyederhanakan kompleksitas dengan menyembunyikan detail implementasi.

FastAPI: Framework Web Modern untuk Python

FastAPI adalah framework web Python yang dirancang untuk membangun API dengan cepat dan efisien.

FastAPI menggunakan type hints Python untuk validasi data otomatis dan dokumentasi API.

Keunggulan FastAPI:

- Performa tinggi: setara dengan NodeJS dan Go berkat Starlette dan Uvicorn.
- Validasi otomatis: menggunakan Pydantic untuk memvalidasi request dan response.
- Dokumentasi otomatis: Swagger UI tersedia di /docs tanpa konfigurasi tambahan.
- Type safety: error terdeteksi lebih awal berkat type hints.
- Async support: mendukung async/await untuk operasi I/O.

Cara membuat endpoint:

1. Buat instance FastAPI: `app = FastAPI()`
2. Definisikan fungsi dengan decorator: `@app.get('/path')` atau `@app.post('/path')`
3. Gunakan Pydantic model untuk request body.
4. Jalankan dengan Uvicorn: `uvicorn main:app --reload`

Pydantic adalah library validasi data yang terintegrasi dengan FastAPI. Dengan Pydantic, kita mendefinisikan schema data sebagai class yang mewarisi `BaseModel`. Setiap field memiliki tipe data yang jelas, dan Pydantic secara otomatis memvalidasi input yang masuk.

Contoh: `class DocumentRequest(BaseModel)` memiliki field `title: str` dan `content: str`. Jika client mengirim data tanpa field `title`, FastAPI otomatis mengembalikan error 422.

Retrieval-Augmented Generation (RAG)

RAG adalah teknik yang menggabungkan pencarian informasi (retrieval) dengan pembuatan jawaban (generation). Alih-alih mengandalkan model bahasa saja, RAG terlebih dahulu mencari konteks yang relevan dari kumpulan dokumen, lalu menggunakan konteks tersebut untuk menyusun jawaban yang lebih akurat.

Komponen Utama RAG:

1. Document Store: tempat menyimpan dokumen asli.
2. Chunking: memecah dokumen menjadi potongan kecil agar pencarian lebih presisi.
3. Embedding: mengubah teks menjadi representasi numerik (vector).
4. Vector Store: menyimpan vector dan memungkinkan pencarian berdasarkan kemiripan.
5. Retriever: mencari chunk yang paling relevan terhadap pertanyaan.
6. Generator: menyusun jawaban berdasarkan konteks yang ditemukan.

Mengapa RAG penting?

- Mengurangi halusinasi: jawaban didasarkan pada dokumen nyata.
- Selalu update: cukup tambah dokumen baru tanpa melatih ulang model.
- Transparan: pengguna bisa melihat sumber jawaban.
- Efisien: tidak perlu fine-tuning model besar.

Alur kerja RAG:

Pertanyaan masuk -> Pertanyaan diubah ke vector -> Cari vector terdekat di store -> Ambil chunk asli -> Susun jawaban dari chunk tersebut -> Kembalikan ke pengguna.

Text Chunking dan Embedding

Text Chunking:

Chunking adalah proses memecah dokumen panjang menjadi potongan-potongan kecil yang disebut chunk. Tujuannya agar setiap potongan cukup pendek untuk diproses, namun tetap mempertahankan konteks yang bermakna.

Strategi chunking yang umum:

- Fixed-size chunking: potong setiap N kata. Sederhana dan konsisten.
- Overlapping chunks: setiap chunk memiliki beberapa kata yang tumpang tindih dengan chunk sebelumnya, agar konteks tidak terputus di batas potongan.
- Semantic chunking: memotong berdasarkan paragraf atau topik (lebih kompleks).

Dalam project ini kita menggunakan fixed-size chunking dengan overlap. Misalnya `chunk_size=30` dan `overlap=5`, artinya setiap chunk berisi 30 kata, dan 5 kata terakhir chunk sebelumnya diulang di awal chunk berikutnya.

Embedding:

Embedding adalah proses mengubah teks menjadi array angka (vector). Teks yang bermakna serupa akan menghasilkan vector yang berdekatan dalam ruang multidimensi.

Dalam project ini, kita menggunakan embedding sederhana berbasis karakter:

- Setiap kata dipetakan ke satu posisi di vector berdasarkan nilai ASCII karakternya.
- Rumus: $\text{posisi} = \text{jumlah}(\text{posisi_karakter} * \text{kode_ascii}) \bmod \text{dimensi_vector}$
- Hasilnya dinormalisasi agar semua vector memiliki panjang yang sama.
- Ini bukan embedding canggih seperti Word2Vec atau BERT, tapi cukup untuk demonstrasi.

FAISS: Pencarian Vector yang Cepat

FAISS (Facebook AI Similarity Search) adalah library open-source dari Meta untuk pencarian kemiripan vector secara efisien. FAISS bisa menangani jutaan vector dengan waktu pencarian yang sangat cepat.

Konsep Kunci FAISS:

- Index: struktur data yang menyimpan vector dan memungkinkan pencarian cepat.
- IndexFlatIP: index paling sederhana yang menghitung Inner Product (dot product).
- Inner Product pada vector yang sudah dinormalisasi sama dengan Cosine Similarity.
- Pencarian dilakukan dengan membandingkan query vector terhadap semua vector di index.

Cara menggunakan FAISS di Python:

1. Import: `import faiss`
2. Buat index: `index = faiss.IndexFlatIP(dimension)`
3. Tambah vector: `index.add(numpy_array)`
4. Cari: `scores, indices = index.search(query_vector, top_k)`

Hasil pencarian berupa:

- scores: array skor kemiripan (semakin tinggi semakin mirip).
- indices: array posisi vector yang ditemukan di index.

FAISS sangat berguna dalam sistem RAG karena memungkinkan pencarian konteks yang relevan secara cepat dari ribuan atau jutaan chunk dokumen.

Streamlit: Membangun Frontend dengan Python

Streamlit adalah framework Python untuk membangun aplikasi web interaktif tanpa perlu menulis HTML, CSS, atau JavaScript. Sangat cocok untuk prototyping dan demo.

Komponen Streamlit yang sering dipakai:

- `st.title()`: menampilkan judul halaman.
- `st.text_input()`: input teks dari pengguna.
- `st.text_area()`: input teks panjang.
- `st.button()`: tombol yang bisa diklik.
- `st.write()`: menampilkan teks atau data.
- `st.tabs()`: membuat navigasi tab.
- `st.sidebar`: area samping untuk kontrol.

Arsitektur Frontend-Backend:

Dalam project ini, Streamlit adalah frontend yang berkomunikasi dengan FastAPI sebagai backend melalui HTTP request. Streamlit mengirim data ke endpoint FastAPI, lalu menampilkan response yang diterima.

Alur komunikasi:

1. Pengguna mengisi form di Streamlit.
2. Streamlit mengirim HTTP POST ke FastAPI (contoh: POST /ask dengan body JSON).
3. FastAPI memproses request menggunakan pipeline RAG.
4. FastAPI mengembalikan response JSON.
5. Streamlit menampilkan hasilnya ke pengguna.

Ini adalah contoh nyata separation of concerns: frontend hanya bertanggung jawab untuk tampilan, sedangkan backend bertanggung jawab untuk logika bisnis dan data.